

运行时环境完整模拟卷 04（题目与完整解析）

目录

卷面说明	2
A 卷题目	2
B 完整解析	5

卷面说明

考试时间：120 分钟

总分：100 分

说明：本卷独立覆盖存储分配、活动树、栈帧、堆管理、引用计数和跟踪式垃圾回收。

所有调用约定、对象图和内存数据均在题面中给出。

A 卷题目

一、存储区、活动树与控制栈（20 分）

某程序的一次执行产生如下动态调用关系，子结点从左到右表示实际调用顺序：

```
main
  f(1)
    g(1)
      g(2)
        h()
```

1. 静态区、栈区、堆区分别存放什么数据，各举一个例子。
2. 写出本次执行的过程调用顺序。
3. 写出过程返回顺序。
4. 当控制流位于 `g(2)` 内部时，哪些活动尚未结束；控制栈从栈底到栈顶有哪些活动记录。
5. 说明活动树的前序/后序遍历与调用/返回顺序的关系。

二、栈帧布局与调用/返回（20 分）

程序：

```
int fun(int a, int b) {
    int c = a - b;
    return c;
}

int main() {
    int i = 100;
    int j = 20;
    int m = fun(i, j);
    return m;
}
```

采用如下明确约定：

机器字长 4 字节；栈向低地址增长。

调用者按从右到左顺序压入参数。

call 指令把返回地址压栈。

被调用者执行 push BP; BP=SP; 再为局部变量分配 4 字节。

函数返回值放入寄存器 R0。

1. 写出调用 fun(i,j) 时，从压入第一个参数到完成函数序言的指令顺序。
2. 以 BP 为基准，写出 c、旧 BP、返回地址、a、b 的地址偏移。
3. 写出执行 c=a-b 和设置返回值的伪汇编。
4. 写出函数尾声和返回过程。
5. 说明返回后 main 如何得到 m=80。

三、堆分配、碎片与分配策略（20 分）

堆中空闲块按地址从低到高排列，初始大小为：

[6, 14, 8, 20]

依次发出三个分配请求：

7, 5, 12

分配时从选中空闲块的低地址端切出所需空间，不考虑对齐和块头开销。

1. 使用 First-Fit，写出每次选中的块及最终空闲块大小。
2. 使用 Best-Fit，写出每次选中的块及最终空闲块大小。
3. 比较两种策略最终最大连续空闲块。
4. 区分内部碎片和外部碎片。
5. 写出存储管理器的空间效率、程序效率和低开销三个评价维度。

四、引用计数与不可达环（20 分）

根和对象引用关系如下：

```
r1 -> A
r2 -> D
A  -> B
B  -> C
C  -> B
D  -> C
```

引用计数只统计根引用和对象字段引用。

1. 写出初始时 A、B、C、D 的引用计数。
2. 执行 `r1=null` 后，按引用计数规则写出回收过程和新计数。
3. 再执行 `r2=null` 后，写出回收过程和新计数。
4. 指出此时哪些对象不可达但不能被引用计数回收。
5. 说明跟踪式垃圾回收为什么能回收它们。

五、标记-清扫、压缩、拷贝与分代（20 分）

根和对象图：

```
R1 -> A      R2 -> F
A -> B, C
B -> D
C -> D, E
D -> B
F -> G
H 无入边
```

对象在堆中按地址排列：

对象	起始地址	大小
A	0	2
H	2	2
B	4	3
C	7	2
D	9	1
E	10	2
F	12	1
G	13	2

1. 从根集出发列出全部可达对象和垃圾对象。
2. 写出标记-清扫两个阶段及清扫结果。
3. 按对象原有顺序向低地址压缩，写出每个存活对象的新起始地址。
4. 说明半空间拷贝回收的 From/To 过程及空间代价。
5. 说明分代回收依据的经验规律，以及记忆集的作用。

B 完整解析

一、存储区、活动树与控制栈解析

存储区：

- 静态区：保存编译期可确定大小和生存期覆盖整个程序的数据，例如全局变量、静态变量、常量和目标代码。
- 栈区：保存随过程调用建立、随返回撤销的活动记录，例如参数、返回地址和局部变量。
- 堆区：保存生命周期不能由调用嵌套确定的动态对象，例如 `malloc/new` 分配的对象。

调用顺序是活动树的前序遍历：

`main, f(1), g(1), g(2), h()`

返回顺序是活动树的后序遍历：

`g(1), g(2), f(1), h(), main`

位于 `g(2)` 时：

- 尚未结束的活动：`main`、`f(1)`、`g(2)`。
- 控制栈从栈底到栈顶：`AR(main)`、`AR(f(1))`、`AR(g(2))`。
- `g(1)` 已经返回，其活动记录已从栈中移除；`h()` 尚未调用。

关系：

- 每个调用在进入子活动前发生，所以调用序列对应前序遍历。
- 一个活动只有在所有子活动返回后才返回，所以返回序列对应后序遍历。
- 当前控制栈正好对应活动树中从根到当前活动结点的路径。

二、栈帧布局与调用/返回解析

调用和序言：

```
push j      ; 参数 b=20, 先压右侧参数
push i      ; 参数 a=100
call fun    ; 压入返回地址, 跳到 fun

fun:
push BP     ; 保存调用者 BP
mov BP, SP  ; 建立当前栈帧基址
sub SP, 4   ; 为局部变量 c 分配 4 字节
```

以 BP 为基准：

```
[BP-4]    c
[BP+0]    旧 BP
[BP+4]    返回地址
[BP+8]    参数 a=100
[BP+12]   参数 b=20
```

函数体：

```
LD  R0, [BP+8]      ; R0=a
SUB R0, [BP+12]     ; R0=a-b=80
ST  [BP-4], R0      ; c=80
LD  R0, [BP-4]      ; 返回值放入 R0
```

函数尾声：

```
mov  SP, BP        ; 释放局部变量
pop  BP            ; 恢复调用者 BP
ret                               ; 弹出返回地址并跳回 main
```

调用者清理两个参数后，把 R0=80 保存到 m：

```
add SP, 8
ST  m, R0
```

三、堆分配、碎片与分配策略解析

First-Fit：

初始：[6,14,8,20]

请求 7：第一个可容纳的是 14，剩余 7
[6,7,8,20]

请求 5：第一个可容纳的是 6，剩余 1
[1,7,8,20]

请求 12：第一个可容纳的是 20，剩余 8
[1,7,8,8]

Best-Fit：

初始：[6,14,8,20]

请求 7：最小可容纳块是 8，剩余 1
[6,14,1,20]

请求 5：最小可容纳块是 6，剩余 1
[1,14,1,20]

请求 12：最小可容纳块是 14，剩余 2
[1,2,1,20]

比较：

First-Fit 最终最大连续空闲块为 8。

Best-Fit 最终最大连续空闲块为 20。

本组请求下 **Best-Fit** 保留了最大的 20 单位块；这不代表所有请求序列中 **Best-Fit** 都更优。

碎片：

内部碎片：已分配块内部因对齐、固定大小类别等原因没有被请求实际使用的空间。

外部碎片：空闲总量足够，但被分散成多个不连续小块，无法满足较大的连续请求。

评价维度：

空间效率：减少总堆占用和碎片。

程序效率：利用缓存和局部性，降低对象访问成本。

低开销：分配和回收算法本身耗时、元数据和暂停尽量小。

四、引用计数与不可达环解析

初始计数：

$RC(A)=1$ ，来自 $r1$ 。

$RC(B)=2$ ，来自 A 和 C 。

$RC(C)=2$ ，来自 B 和 D 。

$RC(D)=1$ ，来自 $r2$ 。

执行 $r1=null$ ：

$RC(A)$ ：1→0，回收 A 。

回收 A 前删除 $A \rightarrow B$ ，使 $RC(B)$ ：2→1。

剩余： $RC(B)=1$ ， $RC(C)=2$ ， $RC(D)=1$ 。

再执行 $r2=null$ ：

$RC(D)$ ：1→0，回收 D 。

回收 D 前删除 $D \rightarrow C$ ，使 $RC(C)$ ：2→1。

剩余： $RC(B)=1$ ， $RC(C)=1$ 。

此时根集为空， B 、 C 均不可达，但 $B \rightarrow C$ 和 $C \rightarrow B$ 使二者计数都为 1，引用计数不能触发回收。

跟踪式回收从根集出发标记对象。根集为空时 B 、 C 都不会被标记，清扫阶段会把二者回收，因此能处理不可达环。

五、标记-清扫、压缩、拷贝与分代解析

从根出发：

R1 可达 A、B、C、D、E。
R2 可达 F、G。
可达集合={A,B,C,D,E,F,G}。
垃圾集合={H}。

标记-清扫：

标记阶段：从 R1、R2 出发遍历引用图，标记 A、B、C、D、E、F、G。
清扫阶段：线性扫描堆，释放未标记的 H，并把 H 的 2 单位空间加入空闲表。

压缩后地址：

对象	旧地址	新地址	大小
A	0	0	2
B	4	2	3
C	7	5	2
D	9	7	1
E	10	8	2
F	12	10	1
G	13	11	2

压缩后连续已用区间为 [0,13)，地址 13 之后为空闲空间。
所有根和对象字段中的地址都必须更新为新地址。

半空间拷贝：

堆分成同样大小的 From 和 To 半空间，正常时只在 From 中分配。
回收时从根出发把可达对象复制到 To，并用转发地址更新引用。
完成后交换 From/To；旧 From 中未复制对象自然成为垃圾。
代价是通常只有约一半堆空间可用于当前分配。

分代回收：

依据“多数对象朝生夕死”的经验规律，新生代容量较小且回收频繁，存活对象晋升到老年代。
老年代回收频率较低。
记忆集记录老年代指向新生代的引用；回收新生代时把这些引用作为额外根，避免扫描整个老年代。